

Batch 与 Momentum

李宏毅《机器学习》2025 · 类神经网络训练不起来怎么办
(二)



基于李宏毅老师课程整理

2026 年 6 月 5 日

视频信息

频道：李宏毅深度学习课堂（Bilibili 搬运）

时长：约 31 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=5>

目录

1 引言：两个让训练跑得动的技巧	3
1.1 先厘清：什么是 Batch 与 Epoch	3
2 为什么要用 Batch：大 batch 与小 batch 的两极对比	4
2.1 平行运算改写了“冷却时间”：大 batch 单次更新并不慢	5
2.2 反转：跑完一个 epoch，大 batch 反而更快	6
3 神奇之处：Noisy 的 Gradient 反面对训练有帮助	7
3.1 实验现象：大 batch 的 training 结果反而更差	8
3.2 为什么小 batch 的 optimization 更好？	8
4 小 batch 连 Testing 都更好：Flat 与 Sharp Minima	9
4.1 解释：好谷底（盆地）与坏谷底（峡谷）	10
4.2 小结：大 batch vs. 小 batch 全面对照	11
4.3 鱼与熊掌能否兼得？	12
5 Momentum (动量)：用惯性冲过 critical point	12
5.1 物理直觉：球不会被小坑卡住	13
5.2 先复习：Vanilla Gradient Descent	13
5.3 加上 Momentum：方向 = 负梯度 + 上一步方向	14
5.4 另一种解读： m^i 是过去所有梯度的加权和	15
6 总结与延伸	16
6.1 核心要点提炼	16
6.2 本节关键概念清单	17
6.3 承上启下	17

1 引言：两个让训练跑得动的技巧

上一节我们弄清楚了“梯度为零”的真相：训练停滞往往不是卡在 local minima，而是卡在 saddle point，而高维空间里其实总有路可走。但“理论上上路”不等于“实务上走得动”——直接算 Hessian、求特征向量来逃离，运算量大到没人真用。

这一节就来补上两个真正实用、运算量又小的训练技巧：Batch（批次）与 Momentum（动量）。它们都能帮助我们对抗 critical point，让 optimization 跑得更顺。

本节的定位

老师原本第一周并不打算讲 batch，因为它“不太符合一般人会采取的直觉做法”，怕讲了反而引出一堆问题。但既然作业的程式里已经用到了，索性先把它讲清楚。本节分两大块：(1) 为什么要用 batch、不同 batch size 的利弊；(2) momentum 如何借“惯性”冲过 critical point。

1.1 先厘清：什么是 Batch 与 Epoch

我们上次没有细讲一个事实：实际算微分时，并不是真的拿所有 data 一起算出总 loss。我们会把所有训练资料切成一个一个的 batch（有人叫 mini-batch，指的是同一件事；课程投影片里写 mini-batch）。

中文字体识别引擎 Labia

Review: Optimization with Batch

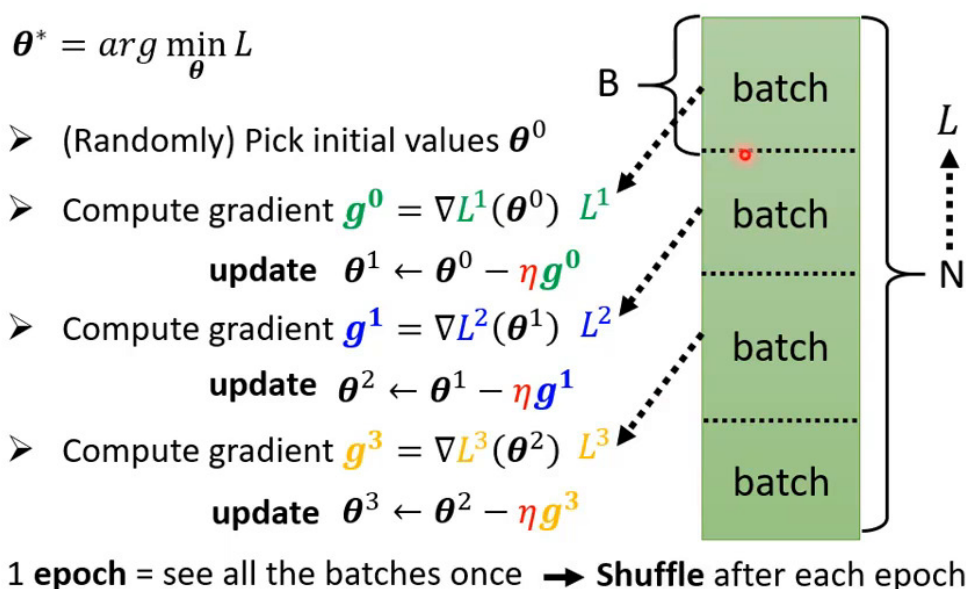


图 1: Optimization with Batch 的完整流程：每次只拿 B 笔资料（一个 batch）算出 L^i 、求 gradient $\mathbf{g}^i$ 、update 一次参数，再换下一个 batch。看完所有 batch 一遍叫做一个 epoch；每个 epoch 开始前会做一次 shuffle，让每个 epoch 的 batch 划分都不同。¹

三个关键术语

- **Batch (批次)**: 每次 update 参数时实际拿来算 loss 与 gradient 的一小撮资料, 大小记为 B 。我们不会拿全部资料一起算, 只拿一个 batch。
- **Epoch (回合)**: 把所有 batch 都看过一遍, 称为一个 epoch。若有 N 笔资料、batch size 为 B , 则一个 epoch 含 N/B 次 update。
- **Shuffle (洗牌)**: 常见做法是每个 epoch 开始前重新随机划分 batch, 于是“哪些资料被分在同一个 batch”每个 epoch 都不一样。

2 为什么要用 Batch: 大 batch 与小 batch 的两极对比

要理解 batch size 的影响, 最好的办法是比较两个最极端的情形。设训练资料共 $N = 20$ 笔:

- **Batch size = N (Full Batch, 等于没用 batch)**: 必须看完全部 20 笔资料, 才能算一次 loss、update 一次参数。
- **Batch size = 1**: 每看一笔资料就 update 一次; 一个 epoch 内会 update 20 次。

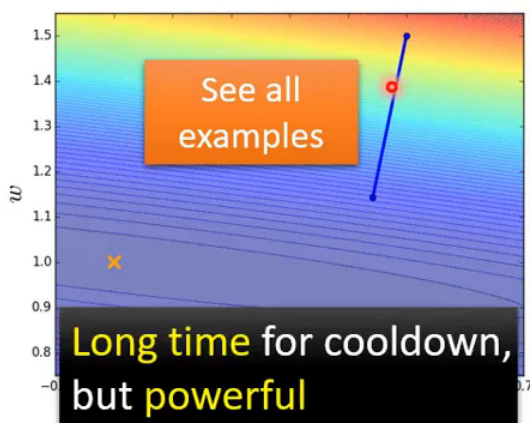
中央研究院学习学堂 Bilibili

Small Batch v.s. Large Batch

Consider 20 examples ($N=20$)

Batch size = N (Full batch)

Update after seeing all the 20 examples



Batch size = 1

Update for each example
Update 20 times in an epoch

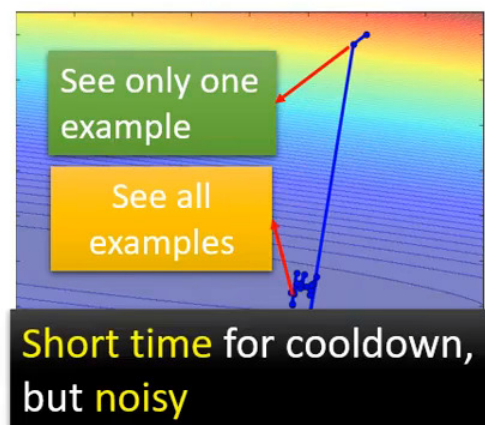


图 2: 两极对比。左 (Full Batch): “看完所有 examples”才更新一次, 蓄力时间长、但这一步走得稳、威力大 (Long time for cooldown, but powerful)。右 (Batch size = 1): 每笔资料更新一次, 冷却时间短、但每步方向曲曲折折、不稳 (Short time for cooldown, but noisy)。

¹ 视频画面时间区间: 00:00:57-00:02:05。

一个生动的比喻：蓄力 vs. 乱枪打鸟

老师把两者比作游戏技能：

- **Full batch**：像是“蓄力”型大招——**技能冷却时间长**（要看完所有资料），但放出来**威力大、方向准**。
- **Batch size = 1**：像是“乱枪打鸟”——**冷却时间短**（看一笔就出手），但每一枪都**不太准**（loss 由单笔资料算出，很 noisy）。

乍看各有优劣。但这个直觉忽略了一个关键因素——平行运算。

2.1 平行运算改写了“冷却时间”：大 batch 单次更新并不慢

“大 batch 冷却时间长”这个直觉，只在**没有考虑平行运算**时才成立。实际上有 GPU 时，一个 batch 里的资料是**平行处理**的——1000 笔资料算 gradient 的时间，**绝不是**1 笔的 1000 倍。

oldest slides: [http://speech.ee.ntu.edu.tw/~tlkagk/courses/MLDS_2015_2/Lecture/DNN%20\(v4\).pdf](http://speech.ee.ntu.edu.tw/~tlkagk/courses/MLDS_2015_2/Lecture/DNN%20(v4).pdf)
old slides: http://speech.ee.ntu.edu.tw/~tlkagk/courses/ML_2017/Lecture/Keras.pdf

Small Batch v.s. Large Batch

- Larger batch size does **not** require longer time to compute gradient

Time for
each update
MNIST: digit
classification

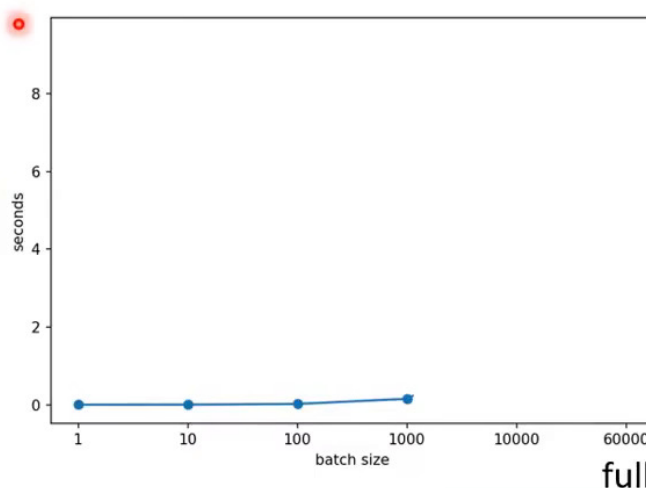


图 3: 在 MNIST 手写数字辨识上实测“每次 update 所需时间”：横轴 batch size 从 1 到 1000，耗时几乎一样（蓝线几乎贴着 0）。结论：Larger batch size does *not* require longer time to compute gradient。

¹ 视频画面时间区间：00:02:51–00:05:20。

¹ 视频画面时间区间：00:06:48–00:07:30。

MNIST: 机器学习界的“果蝇”

MNIST 是手写数字 (0-9) 分类资料集。老师称它为“机器学习的果蝇”——就像生物学家拿果蝇做实验一样，几乎每个刚入门的人第一个尝试的任务就是 MNIST。它小、快、好上手，特别适合用来做这种“batch size vs. 时间”的对照实验。

但平行运算的能力**终究有极限**。当 batch size 大到上万、乃至六万时，GPU 一次要吞下这么多资料，算一个 batch 的时间就会随 batch size 明显上升了。

oldest slides: [http://speech.ee.ntu.edu.tw/~tlkagk/courses/MLDS_2015_2/Lecture/DNN%20\(v4\).pdf](http://speech.ee.ntu.edu.tw/~tlkagk/courses/MLDS_2015_2/Lecture/DNN%20(v4).pdf)
old slides: http://speech.ee.ntu.edu.tw/~tlkagk/courses/ML_2017/Lecture/Keras.pdf

Small Batch v.s. Large Batch

- Larger batch size does **not** require longer time to compute gradient (unless batch size is too large)

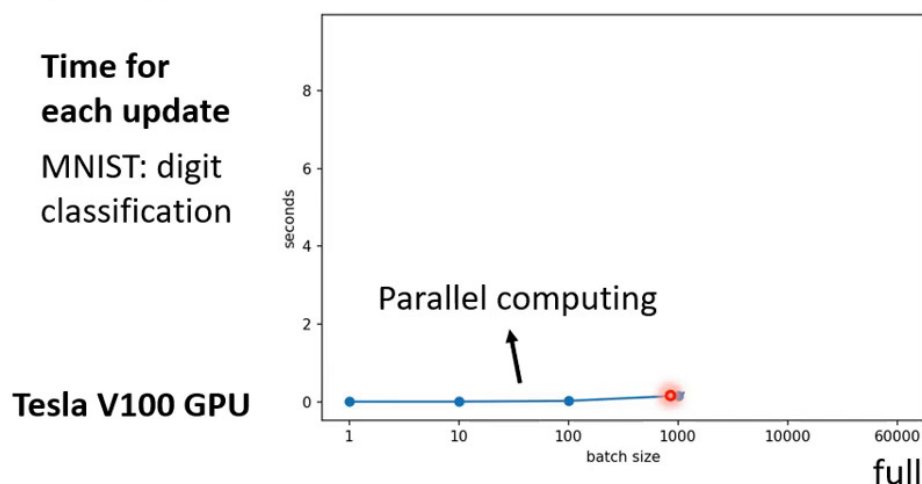


图 4: 补上 batch size = 10000、60000 的情形 (Tesla V100 GPU): 1~1000 区间因**平行运算**几乎持平, 超过约 1000 之后耗时才随 batch size 增长 (*unless batch size is too large*)。即便如此, V100 处理塞了 6 万笔资料的单个 batch, 也能在约十秒内算完 gradient。

硬件的时代演进

老师特意保留了历年的旧投影片: 2015 年用 GTX 760、2017 年用 GTX 980, 跑完 6 万笔资料的一个 batch 要**好几分钟**; 而如今的 V100 只要**十秒**。同一个实验每年都做, 正好见证了 GPU 算力的飞速进步。

2.2 反转: 跑完一个 epoch, 大 batch 反而更快

既然“单次 update 耗时”在 1~1000 区间几乎相同, 那么决定胜负的就变成了一个 epoch 要 update 几次。这里大小 batch 的处境正好**反转**:

¹ 视频画面时间区间: 00:07:48-00:08:25。

Small Batch v.s. Large Batch

- Smaller batch requires longer time for one epoch (longer time for seeing all data once)

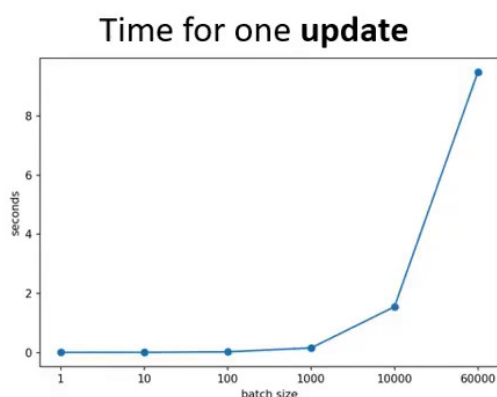


图 5: “跑完一个 epoch (看完所有资料一遍) 所需时间”: 横轴 batch size, 纵轴时间。越小的 batch, 一个 epoch 反而越久——曲线在大 batch 处接近 0、在小 batch 处陡升。这与“单次 update 时间”那张图的趋势恰好相反。

一笔账: 60000 次 vs. 60 次 update

设训练资料 6 万笔:

- Batch size = 1: 一个 epoch 要 update **60000 次**。
- Batch size = 1000: 一个 epoch 只需 update **60 次**。

既然单次 update 的耗时在两者间“根本差不多”，那么 60000 次与 60 次的总时间差距就非常可观。所以在有平行运算的前提下，大 batch 跑完一个 epoch 反而更有效率——这与未考虑平行运算时的直觉正好相反。

到此，大 batch 原本“蓄力慢”的劣势，已经被平行运算抵消甚至反超。看起来大 batch 只剩优势了？且慢——

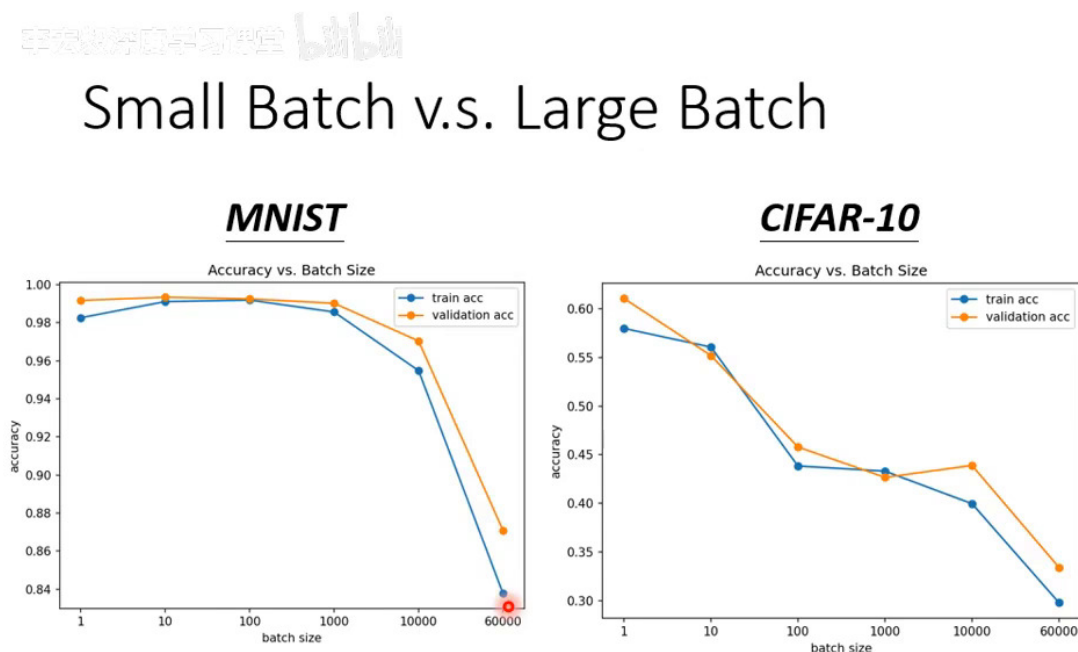
3 神奇之处: Noisy 的 Gradient 反而对训练有帮助

如果大 batch 单次更新不慢、一个 epoch 还更快、方向又更稳定，那它岂不是全面胜出？但真正神奇的地方在于：小 batch 那种 noisy 的 gradient，反而能带来更好的训练结果——这又一次

¹ 视频画面时间区间: 00:09:46-00:10:41。

和直觉相反。

3.1 实验现象：大 batch 的 training 结果反而更差



➤ Smaller batch size has better performance

➤ What's wrong with large batch size? Optimization Fails

图 6: 在 MNIST 与 CIFAR-10 上, 正确率随 batch size 的变化 (蓝: training, 橙: validation)。batch size 越大, 正确率越低。关键观察: 连 training accuracy 都随 batch 增大而下降, 所以这不是 overfitting, 而是 Optimization Fails。

这是 Optimization 问题, 不是 Model Bias、也不是 Overfitting

用的是同一个模型 (同一个 network), 它们能表示的函数集合一模一样, 所以排除 model bias。而 training accuracy 本身就随大 batch 变差——既然在训练集上都没 train 好, 就不是 overfitting, 而是大 batch 时 optimization 出了问题。反过来, 小 batch 的 optimization 结果反而更好。

3.2 为什么小 batch 的 optimization 更好?

一个有力的解释是: 每个 batch 的 loss function 略有差异。

- **Full batch:** 自始至终沿着同一个 loss function L 走, 一旦走到 critical point ($\text{gradient} = 0$), 若不特别去算 Hessian, 就彻底卡住。
- **Small batch:** 第一个 batch 用 L^1 算 gradient, 第二个 batch 用 L^2 而 L^1 与 L^2 的形状不一样。即便在 L^1 上卡住了 ($\text{gradient} = 0$), 换到 L^2 不一定还卡着, 于是又能继续把 loss 降

¹ 视频画面时间区间: 00:12:10–00:13:44。

下去。

Noisy 是一种“扰动救援”

每个 batch 的 loss 略有不同，相当于在 error surface 上不断“换地图”。某张地图上的死路，在下一张地图上可能就通了。所以小 batch 的噪声，恰好成了帮助逃离 critical point 的扰动——noisy 的 update 对 training 反而有帮助。

4 小 batch 连 Testing 都更好：Flat 与 Sharp Minima

更神奇的是：就算想办法（如调大 batch 的 learning rate）把大、小 batch 都 train 到**相同的 training accuracy**，到了 **testing 阶段，小 batch 仍然更好**。这次才是真正的 overfitting 现象——发生在大 batch 上。

On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima

深度学习入门与进阶

<https://arxiv.org/abs/1609.04836>

Small Batch v.s. Large Batch

- Small batch is better on testing data?

	Name	Network Type	Data set
SB = 256	F_1	Fully Connected	MNIST (LeCun et al., 1998a)
	F_2	Fully Connected	TIMIT (Garofolo et al., 1993)
	C_1	(Shallow) Convolutional	CIFAR-10 (Krizhevsky & Hinton, 2009)
	C_2	(Deep) Convolutional	CIFAR-10
LB =	C_3	(Shallow) Convolutional	CIFAR-100 (Krizhevsky & Hinton, 2009)
0.1 x data set	C_4	(Deep) Convolutional	CIFAR-100

Name	Training Accuracy		Testing Accuracy	
	SB	LB	SB	LB
F_1	99.66% ± 0.05%	99.92% ± 0.01%	98.03% ± 0.07%	97.81% ± 0.07%
F_2	99.99% ± 0.03%	98.35% ± 2.08%	64.02% ± 0.2%	59.45% ± 1.05%
C_1	99.89% ± 0.02%	99.66% ± 0.2%	80.04% ± 0.12%	77.26% ± 0.42%
C_2	99.99% ± 0.04%	99.99% ± 0.01%	89.24% ± 0.12%	87.26% ± 0.07%
C_3	99.56% ± 0.44%	99.88% ± 0.30%	49.58% ± 0.39%	46.45% ± 0.43%
C_4	99.10% ± 1.23%	99.57% ± 1.84%	63.08% ± 0.5%	57.81% ± 0.17%

图 7: 出自 *On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima* (arXiv:1609.04836)。作者用 6 个不同 network（含 CNN、全连接网络）在不同资料集上实验：小 batch (SB = 256) 与大 batch (LB = 资料量 × 0.1) 调到**相近的 Training Accuracy**，但 **Testing Accuracy 上小 batch 明显更高**——大 batch 出现了 generalization gap。

¹ 视频画面时间区间：00:15:51–00:17:07。

4.1 解释：好谷底（盆地）与坏谷底（峡谷）

On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima
<https://arxiv.org/abs/1609.04836>

Small Batch v.s. Large Batch

- Small batch is better on testing data?

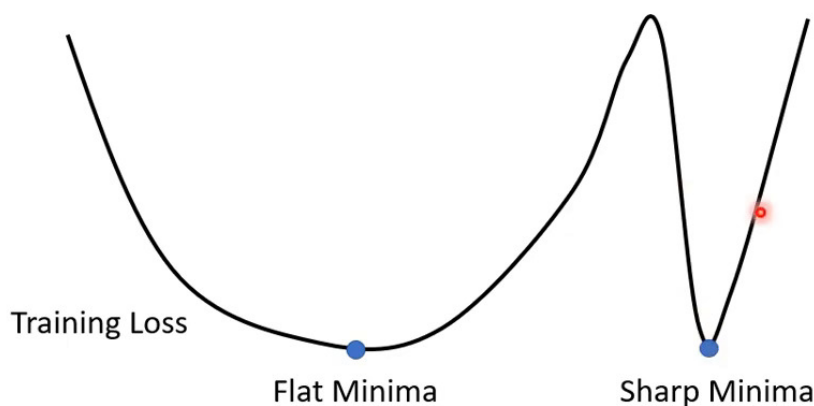


图 8: Training loss 上有多个 local minima, loss 都趋近于零, 但有“好坏”之分: 位于**宽阔盆地**的是 **Flat Minima** (好), 位于**狭窄峡谷**的是 **Sharp Minima** (坏)。

为什么 Flat Minima 泛化更好

Training 与 testing 的 loss 曲线之间存在 **mismatch** (分布不同、或采样不同), 可近似看作把 testing loss 相对 training loss **水平平移**一点。

- 在**平坦盆地 (Flat Minima)** 里: 左右平移一点, loss 几乎不变 $\Rightarrow$ training 与 testing 的结果差不多。
- 在**狭窄峡谷 (Sharp Minima)** 里: 稍一平移, loss **天差地远** $\Rightarrow$ testing loss 暴涨, 泛化差。

小 batch 倾向落入盆地的直觉（尚待研究的猜想）

很多人相信: **大 batch 倾向走进狭窄峡谷, 小 batch 倾向落入宽阔盆地**。直觉是——小 batch 噪声大、每步方向随机, 若峡谷很窄, 一个不小心就**跳出去**了, 只有足够宽的盆地才困得住它; 而大 batch 顺着稳定的 gradient 走, 更容易陷进小峡谷。**注意: 这只是一个解释, 并非人人认同, 仍是开放的研究问题。**

¹ 视频画面时间区间: 00:17:42-00:19:08。

4.2 小结：大 batch vs. 小 batch 全面对照

手撕算法面试题 Bilibili

Small Batch v.s. Large Batch

	Small	Large
Speed for one update (no parallel)	Faster	Slower
Speed for one update (with parallel)	Same	Same (not too large)
Time for one epoch	Slower	Faster 
Gradient	Noisy 	Stable
Optimization	Better 	Worse
Generalization	Better	Worse

图 9: 大、小 batch 的全面对照表。小 batch 在 Gradient 上更 noisy，却在 Optimization 与 Generalization 上更优；大 batch 单次更新（有平行运算时）不慢、一个 epoch 更快。**没有绝对的赢家。**

对照表逐项解读

比较项目	Small Batch	Large Batch
单次 update 时间（无平行运算）	快	慢
单次 update 时间（有平行运算）	相同	相同（除非 batch 过大）
一个 epoch 的时间	慢	快 （占优）
Gradient 方向	Noisy（不稳）	Stable（稳定）
Optimization 表现	更好	较差
Generalization（泛化）	更好	较差

一句话：**batch size 是一个需要你自己去调的 hyperparameter**，两端各有所长。

¹ 视频画面时间区间：00:19:46–00:21:20。

4.3 鱼与熊掌能否兼得？



Have both fish and bear's paws?

- Large Batch Optimization for Deep Learning: Training BERT in 76 minutes (<https://arxiv.org/abs/1904.00962>)
- Extremely Large Minibatch SGD: Training ResNet-50 on ImageNet in 15 Minutes (<https://arxiv.org/abs/1711.04325>)
- Stochastic Weight Averaging in Parallel: Large-Batch Training That Generalizes Well (<https://arxiv.org/abs/2001.02312>)
- Large Batch Training of Convolutional Networks (<https://arxiv.org/abs/1708.03888>)
- Accurate, large minibatch sgd: Training imagenet in 1 hour (<https://arxiv.org/abs/1706.02677>)

图 10: “Have both fish and bear’s paws?”——能否既用大 batch 的平行运算来**加速训练**、又得到小 batch 那样**好的结果**？这是有可能的，已有大量论文探讨（如 76 分钟 train BERT、15 分钟 train ResNet-50、1 小时 train ImageNet 等）。

鱼与熊掌兼得：把 batch 开到极大

“鱼与熊掌”出自《孟子》，喻两件好事难以兼得。这些论文之所以能训练得飞快，正是**把 batch size 开得非常大**（例如第一篇里一个 batch 塞进 3 万笔 sample），用平行运算在极短时间内看过海量资料；同时再辅以**特别的方法**去弥补大 batch 可能带来的劣势。本节不展开，列出 reference 供参考。

5 Momentum（动量）：用惯性冲过 critical point

下课前的第二个技巧是 Momentum，它是另一个**对抗 saddle point 与 local minima**的利器。它的灵感来自**物理世界**。

¹ 视频画面时间区间：00:21:34–00:22:30。

5.1 物理直觉：球不会被小坑卡住

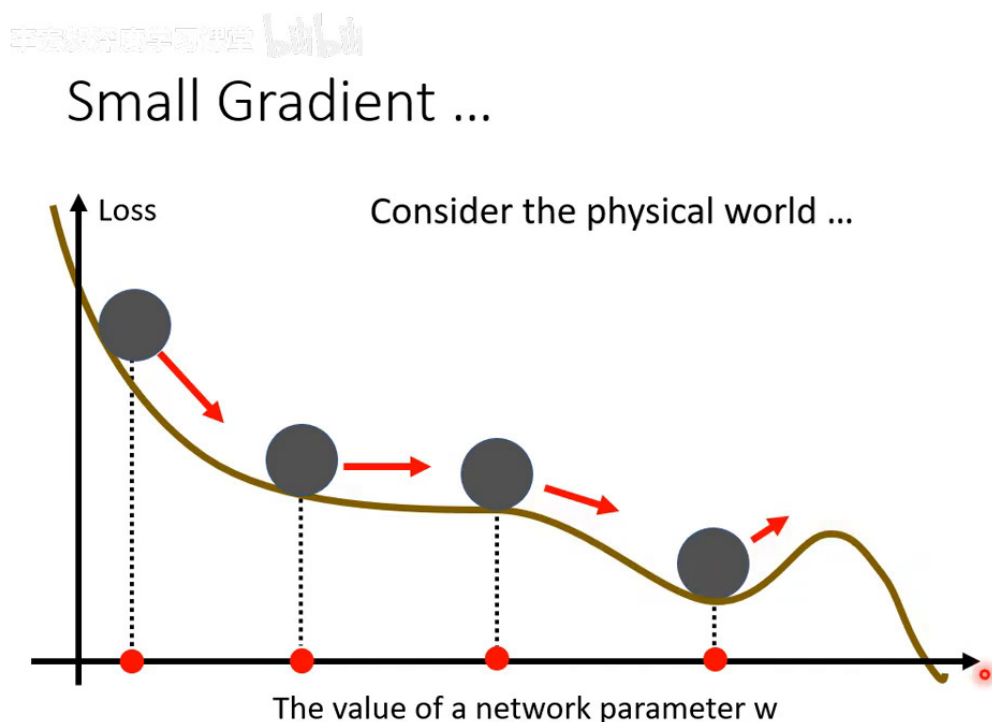


图 11: “Consider the physical world”: 把 error surface 想成真实斜坡、参数想成一颗球。普通 gradient descent 走到 critical point 就停 (gradient = 0); 但物理世界里的球带有惯性, 从高处滚下来, 即便遇到 saddle point 或小的 local minima, 凭动量也可能继续往前、甚至翻过小坡。

从“球”到“算法”

一颗真实的球从斜坡滚下, 不会被 saddle point 或浅坑卡住——惯性会推着它越过。Momentum 要做的, 就是把这种“惯性”引入 gradient descent: 让参数更新时不只听当下的 gradient, 还记得上一步的走向。

5.2 先复习: Vanilla Gradient Descent

一般 (Vanilla) 梯度下降

“Vanilla” 直译是“香草”, 在这里就是“一般、普通”的意思。一般的 gradient descent:

1. 从初始参数 θ^0 出发;
2. 计算 gradient g^0 ;
3. 沿 gradient 反方向 update: $\theta^1 = \theta^0 - \eta g^0$;
4. 到新位置再算 gradient、再沿反方向更新, 如此反复。

它唯一的依据就是当下的 gradient。

¹ 视频画面时间区间: 00:22:50-00:23:52。

5.3 加上 Momentum: 方向 = 负梯度 + 上一步方向

手书体文字: 手书体文字

Gradient Descent + Momentum

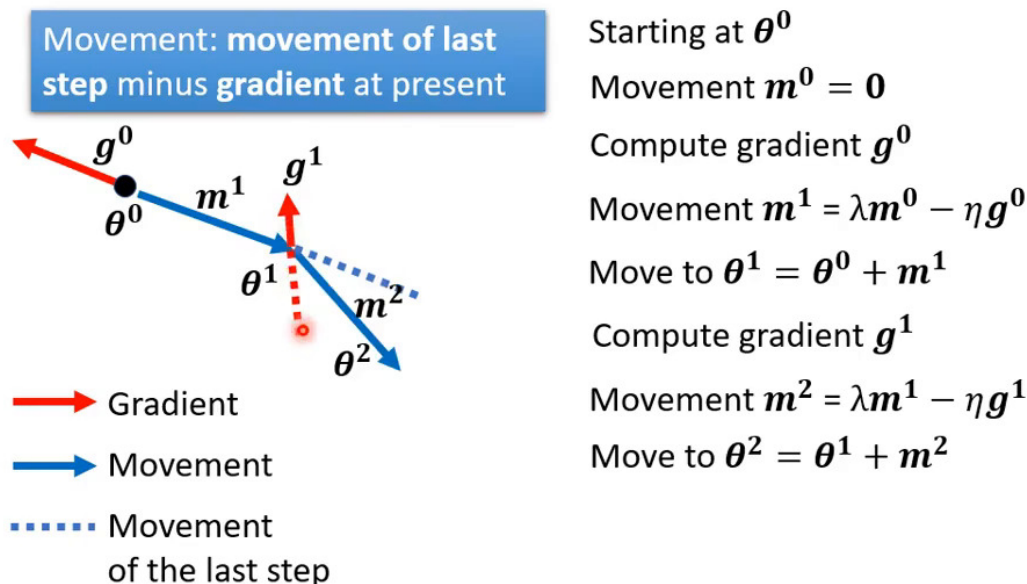


图 12: Gradient Descent + Momentum。每一步的移动量 m^i 不再只由 gradient 决定，而是“上一步的移动”减去“当前的 gradient”： $m^i = \lambda m^{i-1} - \eta g^{i-1}$ 。左图：实际移动（蓝实线）是 gradient 反方向（红）与上一步方向（蓝虚线）的折中（向量和）。

加入 momentum 后的完整流程：

$$\begin{aligned}
 &\text{起点 } \theta^0, \quad \text{初始移动量 } m^0 = 0 \\
 &\text{算 } g^0, \quad m^1 = \lambda m^0 - \eta g^0, \quad \theta^1 = \theta^0 + m^1 \\
 &\text{算 } g^1, \quad m^2 = \lambda m^1 - \eta g^1, \quad \theta^2 = \theta^1 + m^2 \\
 &\vdots
 \end{aligned}$$

各符号含义：

- m^i ：第 i 步的**移动量 (movement)**，即真正用来更新参数的方向。
- g^{i-1} ：当前位置的 gradient； $-\eta g^{i-1}$ 是它的反方向贡献， η 是 **learning rate**。
- λ (lambda)： **动量系数**，衡量“上一步方向”保留多少，是与 η 并列、需要自己调的另一个 hyperparameter。
- 第一步因 $m^0 = 0$ ，方向与普通 gradient descent 相同；从**第二步起**才真正体现出“参考上一步”的差异。

¹ 视频画面时间区间：00:24:49–00:27:01。

5.4 另一种解读： m^i 是过去所有梯度的加权和

手书数学学习课堂 bilibili

Gradient Descent + Momentum

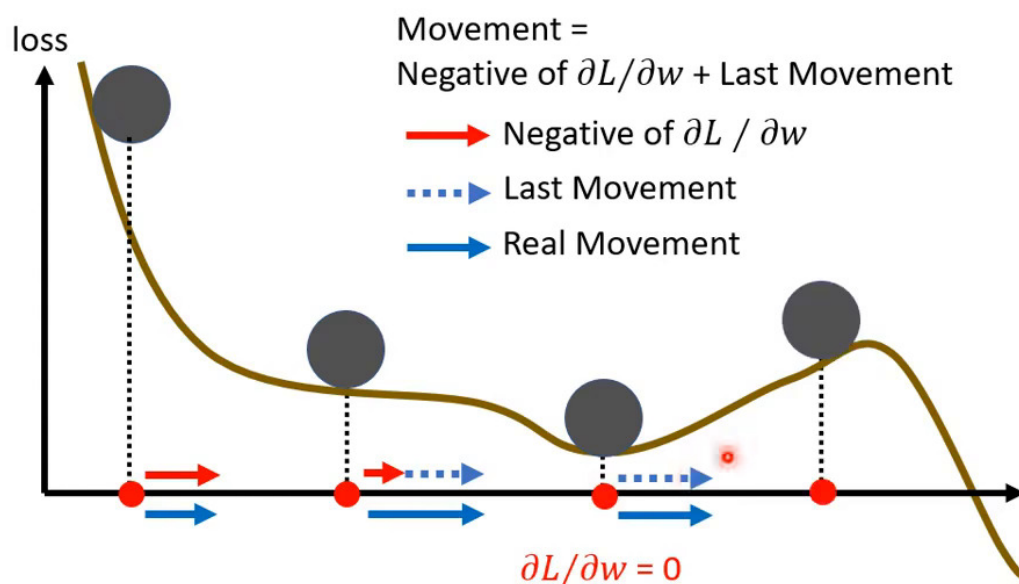


图 13: Momentum 的另一解读与直觉图。展开递推式可知 m^i 是过去所有 gradient 的加权和 (weighted sum)。下方小球图：即便走到 $\partial L / \partial w = 0$ 的 critical point，凭着“上一步方向”（蓝虚线）的惯性，真正的移动（蓝实线）仍能继续向前，甚至翻过小丘走向更好的 local minima。

把递推式逐层展开：

$$\begin{aligned} m^0 &= 0 \\ m^1 &= -\eta g^0 \\ m^2 &= \lambda m^1 - \eta g^1 = -\lambda \eta g^0 - \eta g^1 \\ &\vdots \end{aligned}$$

Momentum 的两种等价解读

1. 递推视角：每一步的移动 = gradient 的反方向 + 上一步移动方向。
2. 展开视角： m^i 是过去所有 gradient $g^0, g^1, \dots, g^{i-1}$ 的加权和——越早的 gradient，被 λ 衰减得越多。

所以加上 momentum 后，update 方向不只考虑当下的 gradient，而是综合了过去的全部 gradient。

¹ 视频画面时间区间：00:27:25–00:30:38。

一个最直观的例子：凭惯性继续走

沿 error surface 滚动：

- 一开始 gradient 指向右，无上一步方向，就单纯往右走一点。
- 走到 local minima / saddle point, gradient = 0, 普通 gradient descent 就此停住。
- 但 momentum 还记得“前一步往右”，于是继续往右，越过这个坑。
- 甚至到了 gradient 指向左（要你往回走）的地方，只要上一步的影响力（动量）够大，仍可能继续往右，翻过小丘，抵达更好的 local minima。

这正是 momentum 可能带来的好处：用惯性对抗 critical point。

6 总结与延伸

本节给出了两个运算量小、却能切实改善训练的技巧，正好补上了上一节“算 Hessian 太贵”留下的缺口。

6.1 核心要点提炼

Batch 部分

1. **Batch / Epoch / Shuffle**: 每次只用一个 batch (B 笔) 算 gradient 并 update; 看完所有 batch 为一个 epoch; 每个 epoch 前 shuffle 重新分组。
2. **平行运算改写直觉**: 有 GPU 时, 1~1000 的 batch 单次 update 耗时几乎相同; 故大 batch 跑完一个 epoch 反而更快。
3. **Noisy 有益**: 小 batch 每个 batch 的 loss function 不同, 能帮助逃离 critical point, optimization 反而更好。
4. **Flat vs. Sharp**: 小 batch 倾向落入平坦盆地 (flat minima), 对 training/testing 的 mismatch 更鲁棒, 故泛化也更好 (注: 此为主流但仍待证实的解释)。
5. **batch size 是 hyperparameter**: 大小各有所长, 需要自己调; “鱼与熊掌”可借特殊方法兼得。

Momentum 部分

1. **物理灵感**: 把参数想成带惯性的球, 球不会被 saddle point 或浅坑卡住。
2. **更新公式**: $\mathbf{m}^i = \lambda \mathbf{m}^{i-1} - \eta \mathbf{g}^{i-1}$, $\boldsymbol{\theta}^i = \boldsymbol{\theta}^{i-1} + \mathbf{m}^i$; 方向 = 负梯度 + 上一步方向。
3. **等价解读**: $\mathbf{m}^i$ 是过去所有 gradient 的加权和, 越旧衰减越多 (系数 λ)。
4. **作用**: 即便当下 gradient 为零, 凭动量仍能继续前进、翻过小丘, 对抗 critical point。

6.2 本节关键概念清单

术语对照表

概念	英文	说明
批次	Batch / Mini-batch	单次 update 实际使用的 B 笔资料
回合	Epoch	看完所有 batch 一遍；含 N/B 次 update
洗牌	Shuffle	每个 epoch 前重新随机划分 batch
全批次	Full Batch	batch size = N ，等于不分 batch
平行运算	Parallel Computing	GPU 同时处理一个 batch 内的资料
噪声梯度	Noisy Gradient	小 batch 方向不稳，但利于逃离 critical point
平坦最小值	Flat Minima	位于宽阔盆地，泛化好
尖锐最小值	Sharp Minima	位于狭窄峡谷，泛化差
泛化差距	Generalization Gap	大 batch 在 testing 上变差的现象
动量	Momentum	更新时叠加“上一步方向”的惯性
动量系数	λ	上一步方向的保留权重（需调）
学习率	Learning Rate η	gradient 反方向的步长（需调）

6.3 承上启下

- **承上**：上一节说“卡住时多半是 saddle point，理论上有路可走，但算 Hessian 太贵”。本节的 small batch 噪声与 momentum 惯性，正是两种运算量极小的“逃生”手段。
- **启下**：“类神经网络训练不起来怎么办”系列还没结束。下一节将讨论自动调整学习率 (Learning Rate) ——当不同参数、不同阶段需要不同步长时，如何让 learning rate 自己适应（如 Adagrad、RMSProp、Adam），进一步解决 error surface 崎岖带来的训练困难。

本节完 · 下一节：类神经网络训练不起来怎么办（三）——自动调整学习率 (Learning Rate)

课程视频：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=5>
